

# TuboEngine

プログラム説明資料

---

制作期間 2025年3月 ～ 2026年5月（約14ヶ月）

制作者 Tsuboki Hayato

技術スタック DirectX 12 / C++17 / HLSL

ジャンル 3Dアクション（弾幕回避）

2026

# 目次

---

1. 作品概要
2. 使用技術・開発環境
3. システム全体構成
4. エンジン層 (engine/)
  - 4.1 DirectX 12 コア
  - 4.2 グラフィックスサブシステム
  - 4.3 ポストエフェクト
  - 4.4 パーティクルシステム
5. ゲームロジック層 (application/)
  - 5.1 プレイヤー
  - 5.2 敵キャラクター・AI
  - 5.3 弾丸システム
  - 5.4 ビヘイビアツリー
6. コアメカニクス：ジャストエヴェイジョン
7. マップ・ステージシステム
8. シーン管理・ステート管理
9. 衝突判定システム
10. 工夫した実装・こだわりポイント
11. 開発を通じて学んだこと

# 1. 作品概要

TuboEngine は DirectX 12 をベースに一から構築した自作 3D ゲームエンジンと、それを用いたアクションゲームの総称です。プレイヤーは敵の弾幕をかわしながら進撃し、ボス戦での精密な回避操作が勝敗を左右します。開発期間は 2025年3月～2026年5月（約14ヶ月）、ソースファイルは 156本にのびります。

| 項目        | 内容                     |
|-----------|------------------------|
| ジャンル      | 3D 弾幕回避アクション           |
| 視点        | 俯瞰（トップダウン）カメラ          |
| 目的        | ステージを進み、ボスを撃破する        |
| コアメカニクス   | ジャストエヴェイジョン（完全回避カウンター） |
| プラットフォーム  | Windows 10/11 64-bit   |
| フレームレート目標 | 60 FPS / 1280×720      |

## ゲームの流れ

- ・ タイトル画面 → チュートリアル → ステージ開始
- ・ 複数チャンクで構成されたマップを進む
- ・ 敵を倒しながらチャンクをクリア
- ・ ボス（CircusEnemy）を撃破でステージクリア
- ・ クリア失敗でゲームオーバー → リトライ

## 2. 使用技術・開発環境

| カテゴリ        | 技術・ツール                             | 用途                         |
|-------------|------------------------------------|----------------------------|
| グラフィックス API | DirectX 12 / DXGI 1.6              | GPU コマンド送信、スワップチェーン        |
| シェーダー言語     | HLSL                               | 頂点・ピクセル・コンピュートシェーダー        |
| 3D アセット読込   | Open Asset Import Library (assimp) | FBX / OBJ モデル読み込み          |
| テクスチャ処理     | DirectXTex                         | DDS / PNG テクスチャロード         |
| GUI (開発用)   | Dear ImGui                         | パラメータ調整・デバッグ表示             |
| JSON 設定     | nlohmann/json                      | ステージ・パーティクル設定ファイル          |
| 言語・標準       | C++17                              | スマートポインタ、ranges、optional 等 |
| IDE / ビルド   | Visual Studio 2022                 | コンパイル・デバッグ                 |

### 3. システム全体構成

プロジェクトは「エンジン層」と「アプリケーション層」に明確に分離されています。エンジン層は再利用可能なフレームワーク、アプリケーション層はゲーム固有ロジックを担います。

| ディレクトリ                  | 役割                                          |
|-------------------------|---------------------------------------------|
| engine/base/            | Win32 ウィンドウ、DirectX 12 デバイス・コマンドリスト管理       |
| engine/Framework/       | メインループ、レンダリングパス制御 (Framework.cpp)           |
| engine/graphic/         | 3D・2D・パーティクル・ポストエフェクト描画サブシステム               |
| engine/scene/           | シーン管理 (SceneManager)、IScene インターフェース        |
| engine/audio/           | サウンド再生 (Audio.cpp)                          |
| engine/input/           | キーボード・マウス入力 (Input.cpp)                     |
| engine/math/            | Vector2/3/4、Matrix、Quaternion 独自実装          |
| application/Character/  | Player・Enemy・BaseCharacter ゲームオブジェクト        |
| application/Bullet/     | 各種弾丸クラス (直進・ホーミング・榴弾)                       |
| application/Stage/      | ステージマネージャー (チャンクストリーミング)                    |
| application/MapChip/    | CSV タイルマップ・衝突グリッド                           |
| application/BT/         | ビヘイビアツリーフレームワーク                             |
| application/Particle/   | パーティクルプリセット・エミッター設定                         |
| application/StageState/ | ステージ内ステート (Ready/Playing/Pause 等)           |
| application/UI/         | HP バー・タイトル UI・ガイド表示                         |
| Resources/              | モデル・テクスチャ・シェーダー・サウンド・CSVマップ                 |
| externals/              | サードパーティライブラリ (assimp、DirectXTex、ImGui、json) |

## 4. エンジン層 (engine/)

### 4.1 DirectX 12 コア

DirectXCommon クラスがデバイス・コマンドキュー・スワップチェーンを一元管理し、ダブルバッファリングによるフレームの同期を行います。SrvManager はシェーダーリソースビュー (SRV) のデスクリプタヒープを管理し、テクスチャや UAV の登録を一元化しています。

| クラス           | 主な責務                                     |
|---------------|------------------------------------------|
| DirectXCommon | D3D12 デバイス・コマンドリスト・フェンス・ダブルバッファ管理        |
| SrvManager    | SRV デスクリプタヒープの確保・解放・インデックス管理             |
| Framework     | メインループ (Init → Update → Draw → Finalize) |
| Order         | レンダリングパスの実行順序制御                          |

### 4.2 グラフィックスサブシステム

| サブシステム   | 主なクラス・機能                                       |
|----------|------------------------------------------------|
| 3D 描画    | Object3d : モデル行列・アニメーション・スキニング                 |
| モデル管理    | ModelManager : assimp 経由 FBX/OBJ 読み込み & キャッシュ  |
| 2D スプライト | Sprite / SpriteCommon : UV アニメーション、スケール・回転     |
| テクスチャ管理  | TextureManager : DirectXTex でロード、SRV 自動登録      |
| テキスト描画   | Font / TextObject / TextManager : ビットマップフォント描画 |
| スカイボックス  | SkyBox / SkyBoxCommon : キューブマップ環境描画            |
| ラインデバッグ  | LineManager / Line : 視野錐台・衝突形状の可視化             |
| オフスクリーン  | OffScreenRendering : MRT (マルチレンダーターゲット)        |

### 4.3 ポストエフェクト

PostEffectManager がオフスクリーンテクスチャに対して複数のエフェクトを順次適用します。各エフェクトは独立した HLSL シェーダーと PSO を持ち、実行時に有効/無効を切り替えられます。

| エフェクト名       | 概要                              |
|--------------|---------------------------------|
| RadialBlur   | 中心点からの放射状ブラー (ジャストエヴェイジョン演出に使用) |
| Bloom        | 高輝度部分を抽出し加算合成する発光効果             |
| GaussianBlur | 水平・垂直 2 パスのガウシアンブラー             |
| Dissolve     | ノイズテクスチャを使ったディゾルブ遷移             |
| Toon         | 輝度を段階的に量子化するトゥーンシェーディング         |

| エフェクト名   | 概要                  |
|----------|---------------------|
| Vignette | 画面端を暗くするビネット効果      |
| VHS      | 走査線・色ずれを模したレトロフィルター |
| Sepia    | セピアトーンカラーグレーディング    |
| Outline  | 法線差分によるエッジ輪郭線描画     |

## 4.4 パーティクルシステム

ParticleManager がエミッター登録辞書（Registry パターン）を持ち、文字列キーで任意のプリセットを生成できます。各エミッターは IParticleEmitter インターフェースを実装し、Update/Draw のみを公開することで拡張性を確保しています。

| プリセット名            | 演出場面              |
|-------------------|-------------------|
| RingEmitter       | 回避・ダッシュ時のリングエフェクト |
| OrbitTrailEmitter | 弾丸や機体の軌跡トレイル      |
| AuraEmitter       | ボスのオーラ・エネルギー演出    |
| ConfettiEmitter   | ステージクリア時の紙吹雪      |
| ExplosionEmitter  | 爆発・被弾エフェクト        |
| FireworkEmitter   | ボス撃破時の花火          |
| MistEmitter       | 環境霧・エリア演出         |
| SparkEmitter      | 金属接触・ガード演出        |

## 5. ゲームロジック層 (application/)

### 5.1 プレイヤー (Player)

Player クラスはゲームオブジェクトの中核で、約 1,000 行の実装を持ちます。64 フレーム分の位置履歴を保持してモーションブラー的なトレイルを表現しつつ、タイトル画面のアイドル時は AutoController が自律的に操作します。

| 機能             | 詳細                                           |
|----------------|----------------------------------------------|
| 移動             | WASD で速度ベクトルを更新、マップ境界でクランプ                   |
| 射撃             | Space / Mouse: クールダウン管理、前方に PlayerBullet を生成 |
| 回避 (Evasion)   | PlayerEvasion クラスへ委譲。無敵フレーム・入力受付ウィンドウを管理     |
| ジャストエヴェイジ      | 完璧なタイミングで回避 → 12 発のホーミング弾を全敵に発射              |
| 位置履歴トレイル       | 64 フレーム分の座標を循環バッファに記録しトレイル描画                 |
| AutoController | タイトルアイドル時: AI が回避タイミング・射撃・移動を制御              |

### 5.2 敵キャラクター・AI (Enemy)

Enemy 基底クラスがビヘイビアツリーと視野コーン検知を持ち、派生クラスが固有の攻撃パターンを実装します。

| クラス              | 特徴・攻撃パターン                                 |
|------------------|-------------------------------------------|
| Enemy (基底)       | BT 駆動 AI、視野角・検知距離カスタマイズ可能、最終目撃地点記憶        |
| CircusEnemy (ボス) | 5 種攻撃 (バースト・スパイラル・クロス・ターゲット・ミサイル)、HP バー演出 |
| RushEnemy        | 「準備」フェーズ後に高速突進、壁バウンドリアクション、ドリルモデルオーバーレイ   |
| MortarEnemy      | 放物線軌道の爆発弾、着弾点を AOE マーカーで予告                |

### 5.3 弾丸システム

| クラス                | 挙動                                |
|--------------------|-----------------------------------|
| PlayerBullet       | 直進、一定距離で消滅                        |
| PlayerCircusBullet | サイン波カオス軌道 → 近接時にターンアシスト強化するホーミング弾 |
| EnemyNormalBullet  | 直進、プレイヤー方向に発射                     |
| CircusBullet       | ボスパラメータと同期したパターン弾                 |
| EnemyMissileBullet | 放物線軌道・着弾半径ダメージ                    |

### 5.4 ビヘイビアツリー (BT/)

BehaviorTree.h/cpp が汎用 BT フレームワークを提供します。Sequence・Selector・Condition・Action の 4 ノードタイプを組み合わせ、各敵クラスが独自の AI ツリーを構築します。



| ノードタイプ    | 動作                          |
|-----------|-----------------------------|
| Sequence  | 子を左から順に実行し、全成功で Success を返す |
| Selector  | 子を左から試し、最初の成功で Success を返す  |
| Condition | bool 関数を評価し、真なら Success     |
| Action    | 具体的な行動（移動・射撃・待機など）を実行       |

## 6. コアメカニクス：ジャストエヴェイジョン

ジャストエヴェイジョンはゲームの中心となる爽快感を生み出すメカニクスです。精確なタイミングでの回避に対して視覚・ゲームプレイ両面の強烈なフィードバックを返します。

### 発動フロー

- Step 1: プレイヤーが回避キーを入力
- Step 2: PlayerEvasion が「受付ウィンドウ」内に弾が接近しているか判定
- Step 3: 条件成立 → 「JUST EVASION!」テキスト表示
- Step 4: RadialBlur エフェクトが最大強度で全画面に展開
- Step 5: 画面上の全弾丸を消去し、プレイヤーに無敵フレームを付与
- Step 6: 12 発の PlayerCircusBullet を全敵に向けて自動発射
- Step 7: ホーミング弾はサイン波カオス軌道で飛翔し、近接時にターンアシスト強化

### PlayerEvasion の状態遷移

| 状態            | 説明                    |
|---------------|-----------------------|
| Idle          | 通常状態。入力待ち             |
| InputWindow   | 回避入力受付中。ジャスト判定を監視     |
| JustEvasion   | ジャスト成立。演出・弾消去・無敵処理を実行 |
| NormalEvasion | 通常回避。短い無敵フレームのみ       |
| Cooldown      | 回避クールダウン中。次の入力を受け付けない |

## 7. マップ・ステージシステム

ステージは複数の「チャンク」に分割されており、StageManager が進行に応じてチャンクをロード・アンロードします。各チャンクはファイルで定義されたタイルマップを持ちます。

CSV

| CSV タイル値     | 意味           |
|--------------|--------------|
| 0 (空白)       | 通路・空きスペース    |
| 1 (Block)    | 壁・障害物 (衝突あり) |
| 2 (Player)   | プレイヤーのスポン位置  |
| 3 (Enemy)    | 敵のスポン位置      |
| 4 (Entrance) | チャンク入口マーカ    |
| 5 (Exit)     | チャンク出口マーカ    |

### チャンクストリーミング

- ・ StageManager がチャンクライフサイクル (生成・更新・破棄) を管理
- ・ プレイヤーが Exit タイルに到達すると次チャンクをロード
- ・ クリア済みチャンク番号を保存し、リトライ時に再開地点を復元
- ・ MapChipField が CSV をパースし、グリッド単位での衝突クエリを提供

## 8. シーン管理・ステート管理

---

### シーンを一覧

| シーン             | 役割                           |
|-----------------|------------------------------|
| TitleScene      | タイトル画面・デモ再生 (AutoController) |
| TutorialScene   | 操作説明・チュートリアル                 |
| StageScene      | メインゲームプレイ                    |
| StageClearScene | クリア結果表示                      |
| GameOverScene   | ゲームオーバー表示                    |
| DebugScene      | 開発用デバッグ画面                    |

### ステージ内ステート (State パターン)

| ステート                 | 役割                      |
|----------------------|-------------------------|
| StageReadyState      | ステージ開始カウントダウン           |
| StagePlayingState    | 通常プレイ。全システムを更新          |
| PauseState           | 一時停止。入力のみ受付             |
| StageClearState      | クリア演出。次チャンクまたはエンディングへ遷移 |
| GameOverState        | ゲームオーバー演出               |
| TutorialState        | ゲーム中チュートリアル表示           |
| StageTransitionState | チャンク間遷移アニメーション          |

### シーン遷移アニメーション (SceneChangeAnimation)

ブロックが画面をランダムなタイミングでタイル状に覆い、イーasing関数で滑らかに展開・収縮するトランジション演出です。約 400 行の実装で独立したアニメーションステートを持ちます。

## 9. 衝突判定システム

CollisionManager が各フレーム全 Collider ペアの総当たり判定を行い、接触時はコールバック（関数ポインタ）を呼び出します。現在は球体コライダーを使用し、型 ID によってどの組み合わせを判定するか制御します。

| クラス              | 役割                           |
|------------------|------------------------------|
| Collider         | 球体コライダーの基底クラス。半径・位置・型 ID を保持 |
| CollisionManager | ブロードフェーズ総当たり、コールバックディスパッチ    |

### 判定ペアと処理

| 判定ペア                         | 処理内容                  |
|------------------------------|-----------------------|
| Player ↔ EnemyBullet         | プレイヤーにダメージ、被弾パーティクル生成 |
| Enemy ↔ PlayerBullet         | 敵にダメージ、ヒットパーティクル生成    |
| Player ↔ Enemy (RadiusCheck) | 互いにダメージ、ノックバック        |
| Bullet ↔ MapBlock            | 弾丸消滅、着弾パーティクル生成       |

## 10. 工夫した実装・こだわりポイント

---

### ■ RadialBlur の動的強度制御

ジャストエヴェイジョン発動時に強度パラメータを時間経過でフェードアウトさせることで、派手な演出後にスムーズにゲームプレイへ戻るよう設計しました。

### ■ ホーミング弾のサイン波カオス

PlayerCircusBullet はサイン波の振幅・周波数をランダムに設定し、不規則な軌道を実現しています。ターゲットに近づくと振幅が減衰して確実に命中するよう制御します。

### ■ 視野コーン検知とビヘイビアツリーの組み合わせ

敵の FOV（視野角・距離）を実行時にカスタマイズ可能にし、「発見 → 警戒 → 追跡 → 攻撃」の状態遷移を BT のノード組み合わせで表現しました。

### ■ チャンクストリーミングによるシームレスな進行

ステージを小単位に分割してロード・アンロードすることで、メモリ使用量を抑えながら長いステージを実現しています。

### ■ 64 フレーム位置履歴トレイル

プレイヤーの過去 64 フレーム分の座標を循環バッファに保存し、補間しながら描画することでモーションブラー的な残像効果を作り出しました。

### ■ デモモード (AutoController)

タイトル画面でアイドル状態になると AI がプレイヤーを操作します。回避タイミング・射撃・移動を適切に制御することで、ゲームの魅力をアピールします。

### ■ ImGui によるリアルタイムパラメータ調整

各システムに ImGui デバッグパネルを設置し、コンパイルなしでパーティクル・AI・ポストエフェクトのパラメータを調整できます。

## 11. 開発を通じて学んだこと

---

### ■ DirectX 12 の低レベル API 習得

D3D12 はリソースバリア・デスク립タヒープ・コマンドリストの明示的管理が必要で、GPU/CPU 同期の仕組みを深く理解することができました。

### ■ ゲームエンジンアーキテクチャの設計

エンジン層とアプリケーション層の分離、シングルトン・ステート・ファクトリ等のデザインパターンの使いどころを実際のプロジェクトで体験しました。

### ■ AI 設計（ビヘイビアツリー）

有限ステートマシンより拡張性が高い BT  
を自作することで、ノード再利用とデバッグのしやすさを実感しました。

### ■ 大規模 C++ プロジェクトの保守性

156 本のソースファイルを 14  
ヶ月管理する中で、命名規則・インターフェース設計・コメント粒度の重要性を学びました。

### ■ パフォーマンスチューニング

パーティクルのオブジェクトプール化、テクスチャキャッシュ、PSOキャッシュなどの最適化手法を実装し、60 FPS 維持を達成しました。

---

以上が TurboEngine のプログラム説明資料です。ご質問があればお気軽にどうぞ。